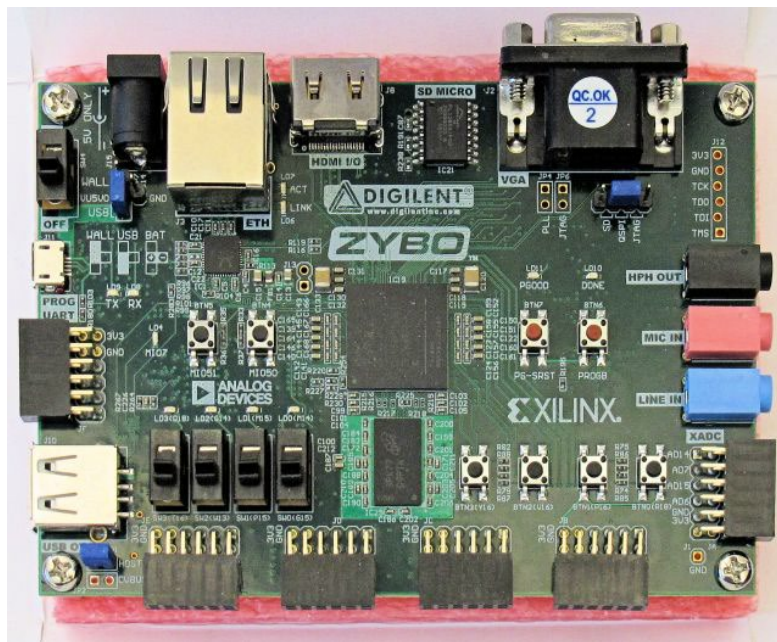


Embedded Real Time Systems



Assignment 2

**Building SoPC designs on a FPGA platform
(Xilinx Zynq - ZYBO)**

Building SoPC designs on a FPGA platform

Description:

In this exercise, you will learn how to develop embedded designs with the ZYBO platform, building customized hardware and software.

You must install Xilinx Vivado version 2017.2 in advance by following the guidelines in “ERTS_Vivado_Installation_2017_2.pdf”.

After installation, you must follow the instructions below:

Setup hardware

Connect ZYBO

- a. Set the power supply jumper to USB so the board can be powered up and laboratory assignments can be carried out using a single micro-USB cable.
- b. Set jumper JP5 to position JTAG.
- c. Connect a micro-USB cable between PROG UART port of ZYBO and PC.

There are two different development boards – the ZYBO and ZYBO Z7 board, which can be used for all exercises; however, remember to use the correct board file, see text below.

IT IS VERY IMPORTANT TO INSTALL SUPPORT FOR THE ZYBO PLATFORM, SEE INSTRUCTION BELOW: (finstallation of Vivado on Windows or Linux)

Copy the *zybo.zip* or *zybo-z7-10.zip* file and extract it in the folder:

<Vivado_2017_2_install_dir>\Vivado\2017.2\data\boards\board_files

This directory is the board files directory and having it in the specified directory will allow you to select the correct Zybo board during the design creation (refer to labs of EmbeddedSystem_labs).

Mandatory exercise for hand-in of assignment: 3, 4, 5, and 7 in journal form. The journal should focus on discussing the results and the obtained learning.

Goals:

- Learn about Xilinx Design Tools, including Vivado, SDK, and HLS.
- Learn the design flow for a system on a programmable chip.
- Learn about configuring the Processing System (PS).
- Learn about the Programmable Logic (PL) and connecting to the PS.
- Learn about the interface driver for USB-UART, GPIO, and TIMER.
- Construct a simple command-line parser to control hardware.
- Learn how to add Custom IPs to your design.
- Learn how to use High Level Synthesis to simulate and generate HDL code.

1.) Building a Complete Embedded SoPC System

Follow the lab guides in the directory **EmbeddedSystem_labs**:

“**lab1_2_EmbeddedSystem.pdf**”, here you will learn to build your first complete embedded SoPC system with the Zync processing system.

“**lab3b_EmbeddedSystem.pdf**”, here you will learn how to add custom IP and BRAM to the system.

2.) Writing Basic Software Application

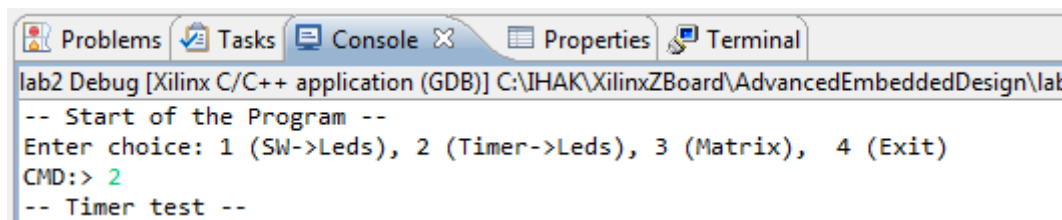
Follow the lab guide in the directory **EmbeddedSystem_labs**:

“**lab4_EmbeddedSystem.pdf**”, here you will learn to write a basic software application and use a custom IP.

3.) Create a Console Application

This exercise builds on the hardware and software platform created in **exercise 2**. Change the Linker Script back to use the **ps7_ram_0_S_AIX_BASEADDR** for the Heap and Stack section.

1. Write a C-application that implements a command language interpreter, controlled via the USB-UART interface. The following commands must be implemented:
 - **1** - Sets the binary value from 0-15 on the red led's by reading switch input (SW0-SW3)
 - **2** - A binary counter using the red LEDs, incrementing the LED pattern every 1 second.



```

lab2 Debug [Xilinx C/C++ application (GDB)] C:\IHAK\XilinxZBoard\AdvancedEmbeddedDesign\lab
-- Start of the Program --
Enter choice: 1 (SW->Leds), 2 (Timer->Leds), 3 (Matrix), 4 (Exit)
CMD:> 2
-- Timer test --
  
```

HINTS:

Input interpreter:

Use the **inbyte()** function to get input character from the USB-UART.

Example:

```

xil_printf("CMD:> ");
/* Read an input value from the console. */
value = inbyte();
skip = inbyte(); //CR
skip = inbyte(); //LF (Skip this line using PuTTY terminal)
switch (value)
{
    case '1':
  
```

To use timer include "XScuTimer.h":

Select the **system.mss** tab in the BSP. Open the [Documentation](#) for **scutimer** on the **Files** link related API functions and data structures are documented for the timer. Example code is listed below for how to initialize the timer and use it.

```

XScuTimer Timer; /* Cortex A9 SCU Private Timer Instance */
#define ONE_SECOND 325000000 // half of the CPU clock speed

// PS Timer related definitions
XScuTimer_Config *ConfigPtr;
XScuTimer *TimerInstancePtr = &Timer;

// Initialize the timer
  
```

```
ConfigPtr = XScuTimer_LookupConfig (XPAR_PS7_SCUTIMER_0_DEVICE_ID);
Status = XScuTimer_CfgInitialize (TimerInstancePtr, ConfigPtr, ConfigPtr->BaseAddr);
If (Status != XST_SUCCESS){
    xil_printf("Timer init() failed\r\n");
    return XST_FAILURE;
}

// Load timer with delay in multiple of ONE_SECOND
XScuTimer_LoadTimer(TimerInstancePtr, ONE_SECOND);
// Set AutoLoad mode
XScuTimer_EnableAutoReload(TimerInstancePtr);

// Start the timer
XScuTimer_Start(TimerInstancePtr);

// Check timer expired
if(XScuTimer_IsExpired(TimerInstancePtr)) {
    // clear status bit
    XScuTimer_ClearInterruptStatus(TimerInstancePtr);
    ....
}

// Stop the timer
XScuTimer_Stop(TimerInstancePtr);
```

4.) Create a Matrix Multiplication

```

// Matrix size
#define MSIZE 4

typedef union {
    unsigned char comp[MSIZE];
    unsigned int vect;
} vectorType;

typedef vectorType VectorArray[MSIZE];

void setInputMatrices(VectorArray A, VectorArray B);
void displayMatrix(VectorArray input);
void multiMatrixSoft(VectorArray A, VectorArray B, VectorArray P);
void multiMatrixHard(VectorArray A, VectorArray B, VectorArray P);
  
```

Use these interfaces for the functions described below

1. Create three global variables of the data structure *vectorArray* called *pInst*, *aInst* and *bTInst*.
2. Implement a function called *setInputMatrices* that fills out the data structures with the input matrix values below:

$$aInst = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}; bTInst = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}^T$$

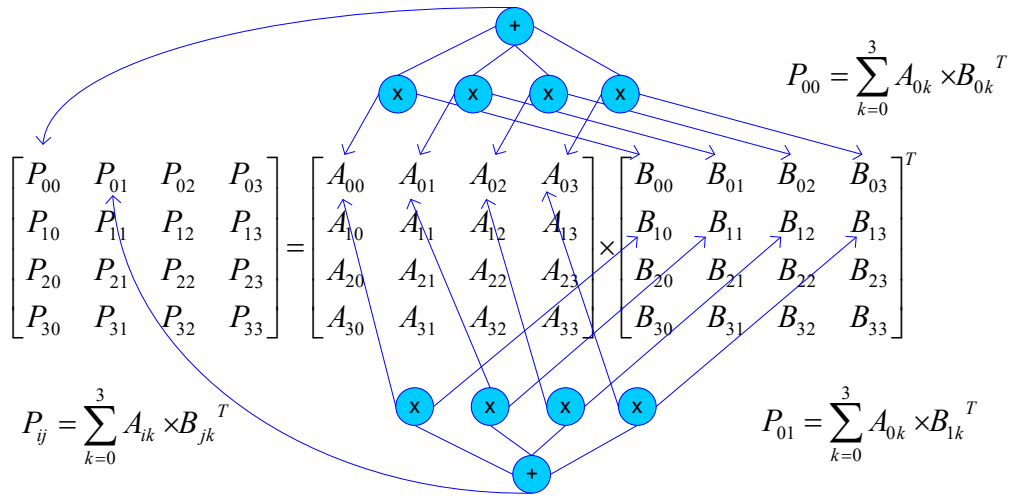
HINT:

$A[0].comp[2] = 3$; sets element on row 0 and column 2 to the value 2 for the matrix A.

3. Implement a function called *displayMatrix* that displays a 4x4 matrix via the USB-UART interface. Test it on matrix *AInst* or *BTinst*.
4. Implement a function called *multiMatrixSoft* that computes the 4x4 matrix product of the expression: $P = A \times B^T$.

HINT:

Use two nested for loops to iterate through all combinations of P_{ij} to calculate the sum of products of each element in P.



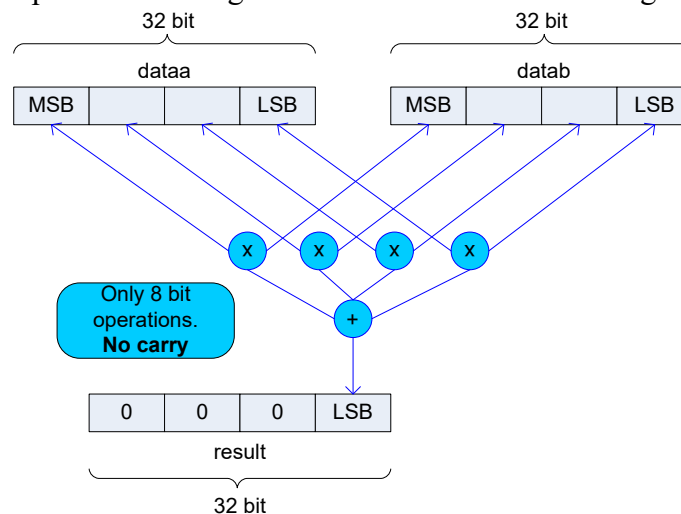
5. Add a command called **mult** that multiplies **AInst** and **BTinst** matrices with the values above. Display the result P and the execution time (in clock ticks) of the function **multiMatrixSoft** in the USB UART window.

5.) Create a Hardware IP Acceleration.

1. Study the implemented VHDL **matrix_ip** component in *matrix_ip_v1_0_S_AXI.vhd* (included in exercise zip file).

All components in VHDL have a number of input and output ports described in the **entity** part of the code. The **architecture** part of the VHDL code describes the implementation the component functionality.

The *matrix_ip* component is adding numbers as illustrated in the figure below.



Remark that all code between **begin** and **end** in the **architecture** part is executed in parallel. VHDL is a strong type language and it is only possible to multiply and add on **unsigned**, **signed** or **integer** types. That's why the `slv_reg0` and `slv_reg1` is casted to **unsigned** and back again to **std_logic_vector** as illustrated in the code snippet below.

```

-- Add user logic here
result1 <= std_logic_vector(unsigned(slv_reg0(31 downto 24)) *
                             unsigned(slv_reg1(31 downto 24)));
result2 <= std_logic_vector(unsigned(slv_reg0(23 downto 16)) *
                             unsigned(slv_reg1(23 downto 16)));
result3 <= std_logic_vector(unsigned(slv_reg0(15 downto 8)) *
                             unsigned(slv_reg1(15 downto 8)));
result4 <= std_logic_vector(unsigned(slv_reg0(7 downto 0)) *
                             unsigned(slv_reg1(7 downto 0)));

i_sum <= std_logic_vector(unsigned(result1(7 downto 0)) +
                          unsigned(result2(7 downto 0)) +
                          unsigned(result3(7 downto 0)) +
                          unsigned(result4(7 downto 0)));

slv_reg2 <= X"000000" & i_sum;
  
```


2. Use the guide “**lab3b_ EmbeddedSystem.pdf**” for how to add a customized IP to the hardware design. Unzip and use the **matrix_ip_1.0.zip** that contains the matrix IP core with user logic to perform multiplication and additions as described above.
3. Create a new function called **multiMatrixHard** that computes the matrix product of the expression: $P = A \times B^T$ but use the new IP core **matrix_ip** to perform part of the matrix multiplication. The C syntax for using the **matrix_ip** is:

```
Xil_Out32(XPAR_MATRIX_IP_0_S_AXI_BASEADDR + MATRIX_IP_S_AXI_SLV_REG0_OFFSET, A[row].vect);  
Xil_Out32(XPAR_MATRIX_IP_0_S_AXI_BASEADDR + MATRIX_IP_S_AXI_SLV_REG1_OFFSET, B[col].vect);  
P[row].comp[col] = Xil_In32(XPAR_MATRIX_IP_0_S_AXI_BASEADDR + MATRIX_IP_S_AXI_SLV_REG2_OFFSET);
```

Here parameters A and B are written to the **matrix_ip** in register REG0 and REG1. The result is returned in register REG2.

HINT:

The multiply and add custom instruction can replace the calculations within the inner loop of the matrix product calculation function. Use the $A[i].vect$ notation of the union structure to read the complete row (four bytes) of the matrix instead of $A[i].comp[j]$ that only reads every matrix element (one byte).

4. Display result P and the execution time (in clock ticks) of the function **multiMatrixHard** in the USB UART window and compare it to **multiMatrixSoft**. The ARM processor (PS) runs at 650 Mhz and the hardware matrix multiplication (PL) is clocking at 100 Mhz. The timer is running at half of the PS clock (325 Mhz). Evaluate the computation speed of software vs. hardware implementation based on measured execution time and clock speeds.

6.) HLS exercise

Follow the lab guides in directory **HLS_labs**:

“**lab1_HLS.pdf**”, here you will learn to simulate and synthesis C-code for HDL code generation. Step 7 of the lab exercise is optional. (Matrix multiplication)

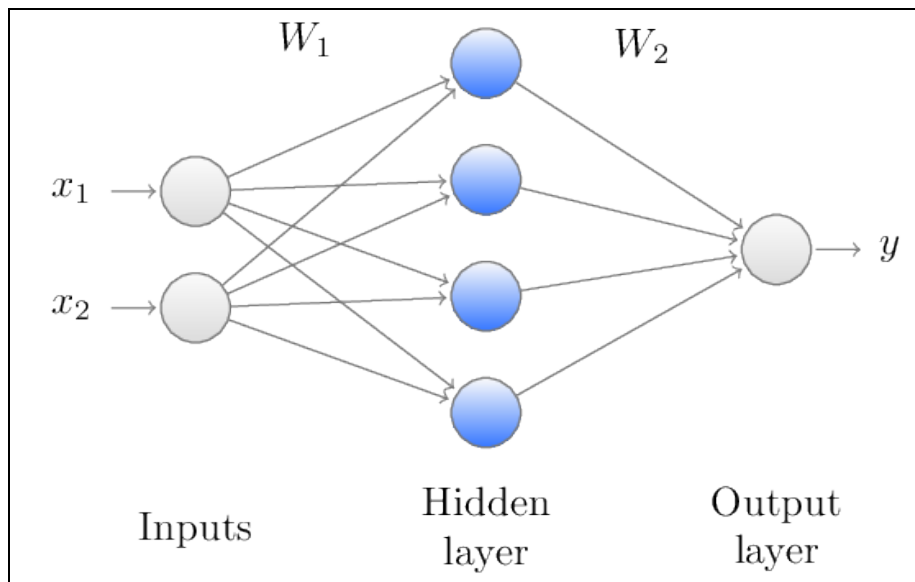
“**lab2_HLS.pdf**”, here you will learn to optimize a design using compiler directives. (YUV image filter)

“**lab4_HLS.pdf**”, here you will learn how to create an IP core that will be inserted into a SoPC design. The design is processing audio stereo music and the HLS IP core performs filtering.

7.) HLS exercise implementing a Neural Network

In this exercise, you will learn how to use C/C++ in writing IP cores controlled by software using the AXI4 Lite interface. (Supporting code is provided in directory **Ex7ANN**)

You should write an IP core in C++ code that should be simulated and tested in Vivado HLS. A prewritten testbench (**testbench.cpp**) is provided that implements a two-layer Neural Network with a hidden layer of neurons and output layer, as shown in the figure below.



The input layer, number of input weights (W_1), and output weights (W_2) and output layers should be possible to configure during simulation before IP synthesis. A ReLU activation function needs to be implemented for neurons in the hidden layer. The IP core of the neural network must be implemented using integer arithmetic (fixed-point). Weights are multiplied by 2^{15} to create integer numbers, and shifting the numbers back must be performed ($\text{neurons}[i] = \text{neuron} \gg \text{BITS}$) after the

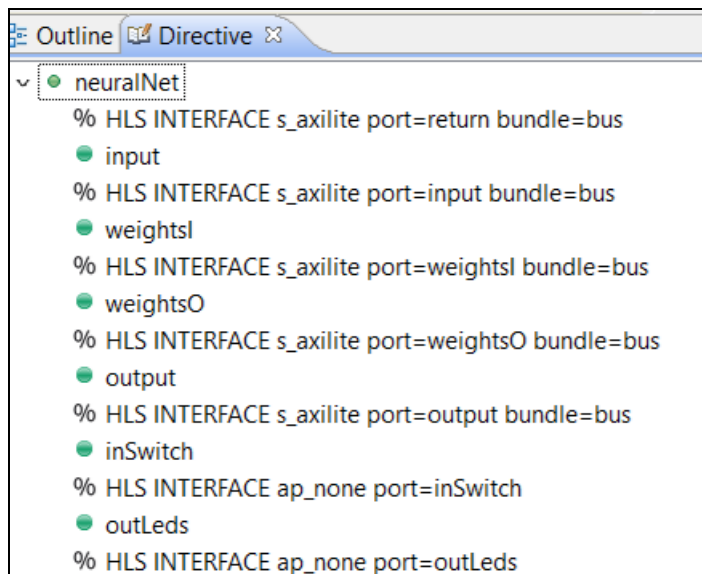
dot product is computed for each neuron. A prototype (*neuralNet.h*) of the C function is shown below:

```

void neuralNet(
    char input[INPUT_LEN],
    short weightsI[WEIGHTS_I_LEN],
    short weightsO[WEIGHTS_O_LEN],
    int output[OUTPUT_LEN],
    ap_int<SL_LEN> inSwitch,
    ap_int<SL_LEN> *outLeds
);
  
```

The inSwitch and outLeds parameters should be connected to the switches and LEDs on the ZYBO board; all other parameters must be mapped to memory-mapped registers (input, weightsO, weightsI, output) that can be controlled from software. The outLeds should be connected to the inSwitch by writing: `*outLeds = inSwitch;`

1. Write the HLS neuralNet IP core and a testbench in C++ code (extension *.cpp).
2. Document and verify the result of simulation and synthesis using Vivado HLS with the provided “testbench.cpp”.
3. Connect the input, weights and output to the AXI4Lite interface using the “INTERFACE” directive.



4. Try to compare a solution by pipelining the two inner loops in neuralNet. What are the differences in terms of latency and area usages (FF, LUT, DSP, and BRAM) for the two solutions? (With and without pipeline)

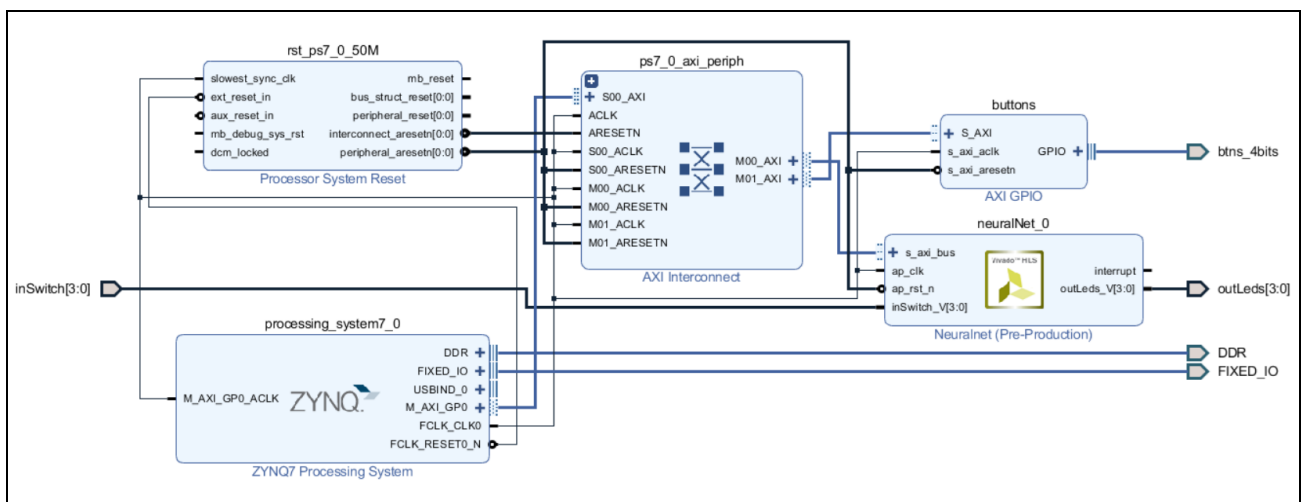
```

% HLS ARRAY_PARTITION variable=neurons complete dim=1
v for Statement
  v ann_label0
    % HLS PIPELINE
  v for Statement
    v ann_label1
      % HLS PIPELINE
  
```

5. Run a C/RTL co-simulation and observe the latency for the solution with the lowest latency.
6. Export the RTL core as VHDL code. (Set version number to 1.0.1)
7. Create a Vivado project (Use guide from **lab1_2_EmbeddedSystem.pdf**) and add the neuralNet IP core and connect it to the LEDS and SWITCHES on the ZYBO board.

In Vivado -> Project Settings -> IP -> Add Repository and browse for the directory:
<project name>\solution1\impl\ip, and it will be possible to select and add the **neuralNet** IP core from **Add IP**.

Add the constraints file **constraints.xdc** that specifies connection to the ZYBO board.



8. Write a program with the Xilinx SDK that verifies the functionality of the IP core and documents the results. (Use inspiration from the “*neuralNetTest.c*” code) Test that the LEDs correspond to the position of the Switches of the ZYBO board.
9. Measure the execution time of a software implementation for neuralNet and compare it to the execution time of accelerating the computation of the neuralNet IP core. Measure separately the writing of data to the IP core (Writing input and weights) and execute the neuralNet IP core. Where are the bottlenecks in the optimized hardware acceleration?